

Uni-App 遮罩层状态转换与日期格式化完整方案

在Uni-App开发中，遮罩层是控制用户交互、展示弹窗内容的核心组件，其状态转换的流畅性直接影响用户体验；而日期格式化则是处理接口数据、展示时间信息的基础功能。本文将整合两者，提供“遮罩层平滑过渡动画+通用日期格式化工具”的实现方案，涵盖基础使用、进阶封装、多场景适配等核心内容。

一、遮罩层状态转换：实现平滑过渡效果

遮罩层的状态转换主要指“显示→隐藏”“隐藏→显示”的切换过程，核心需求是避免状态突变导致的生硬感，通过CSS过渡动画实现透明度与层级的平滑变化。

1. 核心设计思路

- 状态控制**：通过Vue响应式变量（如`isMaskShow`）控制遮罩层的显示/隐藏状态；
- 过渡动画**：使用CSS`transition`属性定义透明度和背景色的过渡效果，时长建议设为0.3s（符合用户视觉感知习惯）；
- 层级管理**：遮罩层z-index需高于页面普通内容，低于弹窗内容区，避免遮挡或被遮挡；
- 交互控制**：支持点击遮罩层关闭、禁止点击穿透、加载状态锁定等实用功能。

2. 基础实现：带过渡效果的遮罩层

适用于简单弹窗场景，实现遮罩层与内容区的联动显示/隐藏，配合平滑过渡动画。

Code block

```
1 <template>
2   <view class="mask-demo-page">
3
4     <button class="open-btn" @click="openMask">打开遮罩层</button>
5
6
7     <view
8       class="base-mask"
9       v-show="isMaskShow"
10      @click="closeMaskByClick"
11    ></view>
12
13
14    <view
15      class="mask-content"
16      v-show="isMaskShow"
```

```
17     >
18     <view class="content-header">
19       <text class="content-title">遮罩层示例</text>
20       <uni-icons
21         type="close"
22         size="28rpx"
23         @click="closeMask"
24       ></uni-icons>
25     </view>
26     <view class="content-body">
27       <text>当前时间: {{ formatDate(new Date()) }}</text>
28     </view>
29     <button class="confirm-btn" @click="closeMask">确认</button>
30   </view>
31 </view>
32 </template>
33
34 <script>
35 import uniIcons from '@dcloudio/uni-ui/lib/uni-icons/uni-icons.vue';
36 // 引入日期格式化工具（下文会实现）
37 import { formatDate } from '@common/utils/dateUtil.js';
38
39 export default {
40   components: { uniIcons },
41   data() {
42     return {
43       isMaskShow: false, // 遮罩层显示状态
44       isMaskLock: false // 遮罩层锁定状态（防止重复点击）
45     };
46   },
47   methods: {
48     // 打开遮罩层
49     openMask() {
50       if (this.isMaskLock) return;
51       this.isMaskLock = true;
52       this.isMaskShow = true;
53       // 解锁：等待过渡动画完成（与CSS transition时长一致）
54       setTimeout(() => {
55         this.isMaskLock = false;
56       }, 300);
57     },
58
59     // 关闭遮罩层
60     closeMask() {
61       if (this.isMaskLock) return;
62       this.isMaskLock = true;
63       this.isMaskShow = false;
```

```

64     setTimeout(() => {
65         this.isMaskLock = false;
66     }, 300);
67 },
68
69 // 点击遮罩层关闭（可选功能）
70 closeMaskByClick() {
71     // 仅当需要点击遮罩关闭时启用
72     this.closeMask();
73 },
74
75 // 日期格式化（直接调用工具函数）
76 formatDate
77 }
78 }
79 </script>
80
81 <style scoped lang="scss">
82 @import "@/common/style/variables.scss";
83
84 .mask-demo-page {
85     padding: 20rpx;
86     min-height: 100vh;
87     display: flex;
88     flex-direction: column;

```

3. 进阶封装：通用遮罩层组件

将遮罩层封装为可复用组件，支持自定义内容、过渡动画、点击行为等，提升开发效率。

(1) 通用遮罩层组件（components/CommonMask/CommonMask.vue）

Code block

```

1  <template>
2    <view class="common-mask-container">
3
4      <view
5        class="mask-bg"
6        :class="{ 'mask-bg--show': isShow }"
7        @click="handleMaskClick"
8        :style="{ backgroundColor: maskColor }"
9      ></view>
10
11
12     <view
13       class="mask-slot"
14       :class="{ 'mask-slot--show': isShow }"

```

```
15         :style="{ zIndex: slotZIndex }"
16     >
17     <slot></slot>
18 </view>
19 </view>
20 </template>
21
22 <script>
23 export default {
24   name: "CommonMask",
25   // 组件属性：支持外部配置
26   props: {
27     isShow: {
28       type: Boolean,
29       required: true,
30       default: false
31     },
32     maskColor: {
33       type: String,
34       default: 'rgba(0, 0, 0, 0.5)' // 默认半透明黑色
35     },
36     closeOnClickMask: {
37       type: Boolean,
38       default: true // 是否点击遮罩关闭
39     },
40     transitionDuration: {
41       type: Number,
42       default: 300 // 过渡动画时长 (ms)
43     },
44     slotZIndex: {
45       type: Number,
46       default: 1000 // 内容区层级
47     },
48     maskZIndex: {
49       type: Number,
50       default: 999 // 遮罩层层级
51     }
52   },
53   methods: {
54     // 点击遮罩层
55     handleMaskClick() {
56       if (this.closeOnClickMask) {
57         // 触发关闭事件，由父组件控制状态
58         this.$emit('close', false);
59       }
60     }
61   }
62 }
```

```

62 }
63 </script>
64
65 <style scoped lang="scss">
66 .mask-bg {
67   position: fixed;
68   top: 0;
69   left: 0;
70   width: 100%;
71   height: 100%;
72   opacity: 0;
73   transition: opacity v-bind(transitionDuration)ms ease; // 绑定动态时长
74   z-index: v-bind(maskZIndex); // 绑定动态层级
75   pointer-events: none; // 隐藏时禁止点击
76 }
77
78 // 遮罩层显示状态
79 .mask-bg--show {
80   opacity: 1;
81   pointer-events: auto; // 显示时允许点击
82 }
83
84 // 内容区样式
85 .mask-slot {
86   position: fixed;
87   top: 50%;
88   left: 50%;

```

(2) 组件使用示例 (pages/advanced-mask/advanced-mask.vue)

Code block

```

1  <template>
2    <view class="advanced-mask-page">
3      <button class="control-btn" @click="isMaskShow = true">打开自定义遮罩
4    </button>
5
6    <CommonMask
7      :isShow="isMaskShow"
8      maskColor="rgba(255, 87, 34, 0.3)"
9      :closeOnClickMask="false"
10     @close="isMaskShow = false"
11   >
12
13     <view class="custom-content">
14       <text class="content-title">自定义遮罩内容</text>

```

```
15         <text class="content-date">格式化日期: {{ formatDate(new Date(), 'yyyy-  
MM-dd HH:mm:ss') }}</text>  
16         <view class="btn-group">  
17             <button class="cancel-btn" @click="isMaskShow = false">取消</button>  
18             <button class="ok-btn" @click="handleConfirm">确认</button>  
19         </view>  
20     </view>  
21 </CommonMask>  
22 </view>  
23 </template>  
24  
25 <script>  
26 import CommonMask from '@/components/CommonMask/CommonMask.vue';  
27 import { formatDate } from '@/common/utils/dateUtil.js';  
28  
29 export default {  
30     components: { CommonMask },  
31     data() {  
32         return {  
33             isMaskShow: false  
34         };  
35     },  
36     methods: {  
37         formatDate,  
38         handleConfirm() {  
39             uni.showToast({ title: '操作成功' });  
40             this.isMaskShow = false;  
41         }  
42     }  
43 }  
44 </script>  
45  
46 <style scoped lang="scss">  
47 .advanced-mask-page {  
48     padding: 20rpx;  
49     min-height: 100vh;  
50     display: flex;  
51     flex-direction: column;  
52     align-items: center;  
53 }  
54  
55 .control-btn {  
56     width: 300rpx;  
57     height: 80rpx;  
58     line-height: 80rpx;  
59     background-color: $theme-secondary;  
60     color: $neutral-white;
```

```

61     border-radius: 40rpx;
62     margin-top: 200rpx;
63 }
64
65 .custom-content {
66     width: 560rpx;
67     background-color: $neutral-white;
68     border-radius: 24rpx;
69     padding: 30rpx;
70     box-shadow: 0 4rpx 20rpx rgba(0, 0, 0, 0.1);
71 }
72
73 .content-title {
74     font-size: 32rpx;
75     font-weight: 600;
76     color: $neutral-black;
77     display: block;
78     margin-bottom: 20rpx;
79 }
80
81 .content-date {
82     font-size: 24rpx;
83     color: $theme-secondary;
84     display: block;
85     margin-bottom: 40rpx;
86     line-height: 40rpx;

```

4. 遮罩层状态转换关键问题解决

(1) 问题1：遮罩层显示时页面可滚动

现象：遮罩层显示后，底部页面仍可上下滚动，影响用户体验。

解决方案：通过动态修改页面body样式禁止滚动，关闭遮罩层后恢复：

Code block

```

1  // 打开遮罩层时禁止滚动
2  openMask() {
3      uni.createSelectorQuery().select('body').boundingClientRect(rect => {
4          document.body.style.overflow = 'hidden';
5      }).exec();
6      this.isMaskShow = true;
7  }
8
9  // 关闭遮罩层时恢复滚动
10 closeMask() {
11     document.body.style.overflow = 'auto';
12     this.isMaskShow = false;

```

```

13  }
14
15  // 适配App端 (无body元素)
16  // #ifdef APP-PLUS
17  const win = plus.webview.currentWebview();
18  win.setStyle({ scrollIndicator: 'none' }); // 隐藏滚动条
19  win.setStyle({ scrollable: false }); // 禁止滚动
20  // #endif

```

(2) 问题2：快速点击导致状态混乱

现象：在过渡动画未完成时快速点击按钮，导致遮罩层状态异常。

解决方案：添加状态锁定变量，在动画执行期间禁止重复操作：

Code block

```

1  data() {
2    return {
3      isMaskShow: false,
4      isTransitioning: false // 动画执行状态锁定
5    };
6  },
7  methods: {
8    openMask() {
9      if (this.isTransitioning) return;
10     this.isTransitioning = true;
11     this.isMaskShow = true;
12     // 动画完成后解锁 (时长与CSS transition一致)
13     setTimeout(() => {
14       this.isTransitioning = false;
15     }, 300);
16   },
17   closeMask() {
18     if (this.isTransitioning) return;
19     this.isTransitioning = true;
20     this.isMaskShow = false;
21     setTimeout(() => {
22       this.isTransitioning = false;
23     }, 300);
24   }
25 }

```

二、日期格式化：通用工具函数封装

日期格式化是将Date对象、时间戳、字符串等格式转换为指定格式（如“2024-05-20”“2024/05/20 13:14:00”）的功能，需支持自定义格式、异常处理、多类型输入适配。

1. 核心需求分析

- **多输入类型支持**：兼容Date对象、时间戳（秒/毫秒）、ISO字符串（如“2024-05-20T05:14:00.000Z”）；
- **自定义格式**：支持“yyyy-MM-dd”“yyyy/MM/dd HH:mm:ss”“MM-dd HH:mm”等常用格式；
- **异常处理**：输入无效日期时返回默认值（如“-”），避免页面报错；
- **补零处理**：月份、日期、小时等小于10时自动补零（如“5月”→“05月”）。

2. 通用日期工具封装（common/utlis/dateUtil.js）

Code block

```
1  /**
2   * 日期格式化工具
3   * @param {Date|Number|String} date - 输入日期 (Date对象/时间戳/ISO字符串)
4   * @param {String} format - 目标格式 (默认: yyyy-MM-dd)
5   * @param {String} defaultValue - 无效日期时的默认返回值 (默认: '-')
6   * @returns {String} 格式化后的日期字符串
7   */
8  export function formatDate(date, format = 'yyyy-MM-dd', defaultValue = '-') {
9    // 1. 处理输入日期, 转换为Date对象
10   let targetDate;
11   if (!date) return defaultValue;
12
13   // 处理时间戳 (支持秒/毫秒)
14   if (typeof date === 'number') {
15     targetDate = date.toString().length === 10
16       ? new Date(date * 1000) // 秒级时间戳转毫秒
17       : new Date(date);
18   }
19   // 处理字符串 (尝试转换为Date)
20   else if (typeof date === 'string') {
21     targetDate = new Date(date);
22   }
23   // 处理Date对象
24   else if (date instanceof Date) {
25     targetDate = date;
26   }
27   // 无效输入
28   else {
29     return defaultValue;
30   }
```

```

31
32 // 验证Date对象有效性
33 if (isNaN(targetDate.getTime())) {
34     return defaultValue;
35 }
36
37 // 2. 提取日期各部分
38 const year = targetDate.getFullYear();
39 const month = targetDate.getMonth() + 1; // 月份从0开始, 需+1
40 const day = targetDate.getDate();
41 const hour = targetDate.getHours();
42 const minute = targetDate.getMinutes();
43 const second = targetDate.getSeconds();
44
45 // 3. 补零函数
46 const padZero = (num) => num.toString().padStart(2, '0');
47
48 // 4. 替换格式字符串
49 const formatMap = {
50     'yyyy': year,
51     'MM': padZero(month),
52     'M': month,
53     'dd': padZero(day),
54     'd': day,
55     'HH': padZero(hour),
56     'H': hour,
57     'hh': padZero(hour % 12 || 12), // 12小时制
58     'h': hour % 12 || 12,
59     'mm': padZero(minute),
60     'm': minute,
61     'ss': padZero(second),
62     's': second
63 };
64
65 // 替换格式 (支持嵌套格式)
66 return format.replace(/yyyy|MM|M|dd|d|HH|H|hh|h|mm|m|ss|s/g, key =>
formatMap[key]);
67 }
68
69 /**
70  * 计算两个日期的差值 (天)
71  * @param {Date|Number|String} startDate - 开始日期
72  * @param {Date|Number|String} endDate - 结束日期
73  * @returns {Number} 日期差值 (正数表示endDate在startDate之后)
74  */
75 export function getDateDiff(startDate, endDate) {
76     const start = new Date(startDate).getTime();

```

```

77     const end = new Date(endDate).getTime();
78     if (isNaN(start) || isNaN(end)) {
79         return 0;
80     }
81     const diffTime = end - start;
82     return Math.floor(diffTime / (1000 * 60 * 60 * 24));
83 }
84
85 /**
86  * 获取指定日期的星期
87  * @param {Date} date 日期对象
88  * @returns {string} 星期字符串
89  */

```

3. 日期格式化使用示例

Code block

```

1  import { formatDate, getDateDiff, getWeekDay } from
    '@common/utils/dateUtil.js';
2
3  // 1. 格式化Date对象
4  const now = new Date();
5  formatDate(now); // 输出: 2024-05-20 (默认格式)
6  formatDate(now, 'yyyy/MM/dd HH:mm:ss'); // 输出: 2024/05/20 13:14:00
7  formatDate(now, 'MM-dd HH:mm 周E', ['日', '一', '二', '三', '四', '五', '六']); // 输出: 05-20 13:14 周一
8
9  // 2. 格式化时间戳 (秒级)
10 const timestamp = 1716184440;
11 formatDate(timestamp, 'yyyy-MM-dd HH:mm'); // 输出: 2024-05-20 13:14
12
13 // 3. 格式化ISO字符串
14 const isoStr = '2024-05-20T05:14:00.000Z';
15 formatDate(isoStr, 'yyyy年MM月dd日'); // 输出: 2024年05月20日
16
17 // 4. 计算日期差值
18 const start = '2024-05-01';
19 const end = '2024-05-20';
20 getDateDiff(start, end); // 输出: 19
21
22 // 5. 获取星期
23 getWeekDay(now); // 输出: 周一

```

4. 常见格式化场景适配

(1) 场景1: 接口返回时间戳处理

接口通常返回秒级或毫秒级时间戳，直接传入工具函数即可自动适配：

Code block

```
1 // 接口返回数据
2 const apiData = {
3   createTime: 1716184440, // 秒级时间戳
4   updateTime: 1716184440000 // 毫秒级时间戳
5 };
6
7 // 格式化展示
8 formatDate(apiData.createTime, 'yyyy-MM-dd HH:mm:ss'); // 2024-05-20 13:14:00
9 formatDate(apiData.updateTime, 'yyyy-MM-dd'); // 2024-05-20
```

(2) 场景2：表单提交日期格式化

将选择器选择的日期转换为接口需要的格式（如“yyyy-MM-dd”）：

Code block

```
1 <template>
2   <view>
3     <uni-datetime-picker
4       v-model="selectDate"
5       @change="handleDateChange"
6     ></uni-datetime-picker>
7   </view>
8 </template>
9
10 <script>
11 import { formatDate } from '@common/utils/dateUtil.js';
12 export default {
13   data() {
14     return {
15       selectDate: new Date()
16     };
17   },
18   methods: {
19     handleDateChange(e) {
20       // 转换为接口需要的格式
21       const formatDateStr = formatDate(e.value, 'yyyy-MM-dd');
22       // 提交表单
23       this.submitForm({ date: formatDateStr });
24     }
25   }
26 }
27 </script>
```

(3) 场景3：相对时间显示（如“今天”“昨天”）

结合日期差值函数，实现相对时间与绝对时间的切换：

Code block

```
1  /**
2   * 相对时间格式化
3   * @param {Date|Number|String} date - 输入日期
4   * @returns {String} 相对时间 (今天/昨天/MM-dd)
5   */
6  export function formatRelativeDate(date) {
7      const targetDate = new Date(date);
8      if (isNaN(targetDate.getTime())) return '-';
9
10     const today = new Date();
11     today.setHours(0, 0, 0, 0);
12
13     const yesterday = new Date(today);
14     yesterday.setDate(yesterday.getDate() - 1);
15
16     const target = new Date(targetDate);
17     target.setHours(0, 0, 0, 0);
18
19     const diff = getDateDiff(today, target);
20
21     if (diff === 0) {
22         return `今天 ${formatDate(date, 'HH:mm')}`;
23     } else if (diff === -1) {
24         return `昨天 ${formatDate(date, 'HH:mm')}`;
25     } else if (diff > 0 && diff <= 7) {
26         return `${getWeekDay(date)} ${formatDate(date, 'HH:mm')}`;
27     } else {
28         return formatDate(date, 'yyyy-MM-dd HH:mm');
29     }
30 }
31
32 // 使用示例
33 formatRelativeDate(new Date()); // 今天 13:14
34 formatRelativeDate('2024-05-19'); // 昨天 00:00
35 formatRelativeDate('2024-05-15'); // 周三 00:00
```

三、综合实战：遮罩层与日期格式化结合场景

以“日期选择弹窗”为例，实现遮罩层平滑显示/隐藏，配合日期选择器与格式化功能，完成日期选择与提交流程。

Code block

```
1  <template>
2    <view class="date-picker-page">
3
4      <view class="date-display" @click="isMaskShow = true">
5        <text class="display-label">选择日期: </text>
6        <text class="display-value">{{ selectedDate || '点击选择' }}</text>
7        <uni-icons type="right" size="24rpx" color="#999"></uni-icons>
8      </view>
9
10
11    <CommonMask
12      :isShow="isMaskShow"
13      @close="isMaskShow = false"
14    >
15      <view class="date-picker-content">
16        <view class="picker-header">
17          <text class="header-title">选择日期</text>
18          <button class="confirm-picker-btn" @click="confirmDate">确认</button>
19        </view>
20        <uni-datetime-picker
21          v-model="pickerDate"
22          type="date"
23          :start="startDate"
24          :end="endDate"
25        ></uni-datetime-picker>
26      </view>
27    </CommonMask>
28  </view>
29 </template>
30
31 <script>
32 import CommonMask from '@components/CommonMask/CommonMask.vue';
33 import uniIcons from '@dcloudio/uni-ui/lib/uni-icons/uni-icons.vue';
34 import uniDatetimePicker from '@dcloudio/uni-ui/lib/uni-datetime-picker/uni-
35   datetime-picker.vue';
36 import { formatDate } from '@common/utils/dateUtil.js';
37
38 export default {
39   components: { CommonMask, uniIcons, uniDatetimePicker },
40   data() {
41     const now = new Date();
42     return {
43       isMaskShow: false,
44       pickerDate: now, // 选择器绑定日期
45       selectedDate: '', // 已选格式化日期
```

```

45         startDate: new Date(now.getFullYear() - 1, now.getMonth(),
now.getDate()), // 开始日期 (1年前)
46         endDate: now // 结束日期 (今天)
47     };
48 },
49     methods: {
50         // 确认选择日期
51         confirmDate() {
52             // 格式化选择的日期
53             this.selectedDate = formatDate(this.pickerDate, 'yyyy-MM-dd 周E');
54             // 关闭遮罩层
55             this.isMaskShow = false;
56             // 提交日期 (实际场景中调用接口)
57             this.submitSelectedDate(this.selectedDate);
58         },
59
60         // 提交已选日期
61         submitSelectedDate(date) {
62             uni.request({
63                 url: "https://your-server/api/submit-date",
64                 method: "POST",
65                 data: { date },
66                 success: (res) => {
67                     if (res.data.code === 0) {
68                         uni.showToast({ title: '日期提交成功' });
69                     }
70                 }
71             });
72         }
73     }
74 }
75 </script>
76
77 <style scoped lang="scss">
78 .date-picker-page {
79     padding: 20rpx;
80     min-height: 100vh;
81     background-color: $neutral-bg;
82 }
83
84 .date-display {
85     display: flex;
86     align-items: center;

```

四、总结：核心要点与最佳实践

1. 遮罩层状态转换核心要点

1. **动画平滑**：使用`transition`属性定义透明度、缩放等过渡效果，时长控制在0.2-0.3s；
2. **层级清晰**：遮罩层z-index设为999，内容区设为1000，避免与页面其他元素层级冲突；
3. **交互可控**：添加点击遮罩关闭、状态锁定、禁止页面滚动等功能，提升体验；
4. **组件复用**：将遮罩层封装为通用组件，通过props传递配置，减少重复代码。

2. 日期格式化核心要点

1. **输入兼容**：支持Date对象、时间戳、字符串等多种输入类型，自动转换处理；
2. **格式灵活**：通过格式字符串替换实现自定义格式，满足不同场景需求；
3. **异常容错**：对无效日期输入返回默认值，避免页面报错；
4. **工具化封装**：将格式化函数封装为全局工具，在全项目复用，便于维护。



实际开发中，可结合Uni-UI的`uni-popup`组件简化遮罩层开发，其内置了完善的过渡动画与多端适配；日期格式化也可使用第三方库（如date-fns），但轻量场景下自定义工具函数更高效。两者结合时，需确保遮罩层状态转换与日期选择逻辑联动顺畅，提升用户操作体验。

（注：文档部分内容可能由 AI 生成）